

CPU 性能分析：perf 与 Intel VTune

目 录

1 引言：大型程序不能靠猜	2
2 两套 CPU 环境	2
3 perf 工具总览	2
3.1 perf stat：硬件计数器	2
3.2 perf record：采样热点与调用栈	3
3.3 perf report：查看报告	3
4 采样与追踪	4
5 perf 分析要点	4
6 VTune 工具：V100 节点 host 端分析	4
6.1 完整调用链	5
6.2 Hotspots 分析	5
6.3 Bottom-up 调用树	5
6.4 Flame Graph 与 Threads 视图	5
6.5 多 rank 结果下载	5
7 VTune 回答的问题	5
8 关键误区：host 等待不等于 kernel 时间	6
9 在 AMSS-NCKU 中应用	6
10 本章你将学会	7
11 要点速查	7
12 小结	8

1 引言：大型程序不能靠猜

AMSS-NCKU 数值相对论程序是一个体量很大的代码：它的调用链从 Python 驱动脚本出发，经过 **TwoPuncture**（初始数据生成）模块，再通过 `mpirun` 启动多个 rank，每个 rank 内部跑 ABE 或 ABEGPU 的 BSSN 方程演化。面对这样一条横跨 Python、C、MPI、CUDA 的调用链，你不可能靠“读代码”猜出时间花在哪。你猜 `bssn_rhs` 慢，可能其实是 MPI 等待慢；你猜 GPU kernel 慢，可能其实是 host 在 `cuCtxSynchronize` 上空等。**Profiler**（性能分析器）的作用就是把“我觉得”变成“我看到”。

本章介绍 Lab4 的两套 CPU 分析工具：在 Huawei Kunpeng 920B（ARM 架构）上用 `perf` 分析 task1 的 ABE 演化，在 V100 节点的 x86 host 上用 Intel VTune 分析 task2 的 ABEGPU host 端。

直觉 Profiler 的价值不在于告诉你“哪里慢”，而在于告诉你“为什么慢”。在 AMSS-NCKU 这种多模块程序里，“哪里慢”往往出乎意料：你以为热点在 BSSN 右端项计算，结果发现 host 一直在等 GPU，或者某个 rank 比别的 rank 多算了一倍网格。没有 profiler，你会把时间浪费在优化一个不占多少比例的函数上。

2 两套 CPU 环境

Lab4 有两个 CPU 评测环境，对应两个不同的分析工具链：

任务	CPU 平台	架构	分析工具
Task1 ABE 演化	Huawei Kunpeng 920B	ARM v8.2	<code>perf</code>
Task2 ABEGPU	V100 节点 host	x86	Intel VTune

Table 1: Lab4 的两套 CPU 环境与对应工具

为什么不用同一个工具？因为 `perf` 是 Linux 内核自带的轻量采样器，依赖内核 PMU（Performance Monitoring Unit）接口，在 ARM 上同样可用；而 Intel VTune 是 Intel 自家工具，对 x86 的微架构计数器支持更全，也能更好地解析 host 上的 MPI 和 CUDA 调用。在 V100 节点上，我们要分析的恰好是 host 端的调用链，所以 VTune 更合适。

注意两套环境分析的对象不同：*task1* 用 `perf` 分析的是纯 CPU 程序本身；*task2* 用 VTune 分析的是 ABEGPU 的 **host 端**，也就是 CPU 侧，GPU 侧要等到下一章用 *Nsight Systems* 才能看清。

3 perf 工具总览

`perf` 是 Linux 内核附带的一组子命令，常用的有三个：`perf stat` 读硬件计数器，`perf record` 采样热点和调用栈，`perf report` 查看报告。我们逐个展开。

3.1 perf stat：硬件计数器

`perf stat` 读取 CPU 的硬件性能计数器（PMU），给出程序运行期间累计的统计值。它不抓调用栈，开销极小，适合先跑一遍拿到“宏观体检报告”。

```
perf stat -d ./run.sh
```

-d 表示 detailed mode，会额外显示 cache miss、TLB miss 等细分计数器。典型输出片段：

```
1234.567890 seconds time elapsed
4.0 CPUs utilised
10 234 567 890 instructions          # 0.60 insn per cycle
2 345 678 901 branches              # 1.9 G/sec
12 345 678 branch-misses            # 0.53% of all branches
3 456 789 012 L1-dcache-load-misses  # 28.4% of all L1-dcache loads
56 789 012 dTLB-load-misses         # 0.46% of all dTLB loads
```

直觉 把这些数字读成“体检指标”就好：IPC（insn per cycle）告诉你 CPU 是否在吃饱，cache miss 比例告诉你是不是在等内存，branch miss 比例告诉你分支预测是否在拖后腿。一台健康的 CPU 在计算密集代码上 IPC 应该接近 2 甚至更高；如果只有 0.6，说明流水线大部分时间在停顿。

关键计数器含义如下：

计数器	回答什么
IPC (insn per cycle)	CPU 是否吃饱，停顿多不多
L1-dcache-load-misses	数据是否在缓存里
dTLB-load-misses	地址翻译是否成为瓶颈
branch-misses	分支预测是否浪费

Table 2: perf stat 关键计数器

3.2 perf record: 采样热点与调用栈

perf stat 只给累计数字，不告诉你“哪个函数”慢。perf record 通过周期性采样 CPU 上正在执行的指令地址，把时间归属到函数。加上 --call-graph dwarf 选项可以同时抓调用栈，这样你不仅能看到热点函数，还能看到是谁调用了它。

```
perf record --call-graph dwarf -- ./run.sh
```

--call-graph dwarf 使用 DWARF 调试信息展开调用栈，比默认的 FP（frame pointer）模式更准确，但开销略大。

要看到带函数名和源码行的报告，程序必须用 -g 编译（保留调试信息）。如果只看到十六进制地址，多半是没开 -g 或是库被 strip 过。在 AMSS-NCKU 中，确认 ABE、ABEGPU 和 TwoPunctureABE 这几个可执行文件都带了调试符号。

3.3 perf report: 查看报告

采样结束后会生成 perf.data，用 perf report 进入交互式界面查看：

perf report

默认按“overhead”降序排列函数，你可以在界面里展开调用栈（按+），切换到调用图视图。如果想在脚本里提取结果，可以用 `perf report --stdio > report.txt` 把报告导出成纯文本。

4 采样与追踪

`perf stat` 和 `perf record` 都属于采样（sampling）：它们每隔固定周期（或事件计数达到阈值）抓一次“当前在执行什么”，开销小但结果是统计性的。采样的精度取决于采样频率和运行时长。

另一种方式是追踪（tracing）：把每一帧感兴趣的事件都带上时间戳记录下来。Linux 的 `perf` 也能做轻量追踪（如 `perf sched`、`ftrace`），而 VTune 的时间线视图本质上也是一种追踪。

直觉 采样像“每隔一秒拍一张照片”，能告诉你“谁出现最多”，但不能告诉你“谁先谁后”。追踪像“全程录像”，能告诉你“几点几分谁在等谁”。诊断负载不均衡、同步等待这种**时序相关**的问题，必须靠追踪；诊断“谁占比例最大”这种**统计相关**的问题，采样就够了。

在 Lab4 中，rank 间负载不均衡就属于时序问题：某个 rank 算完在 `MPI_Waitall` 上空等其它 rank。这种问题采样能看到“`MPI_Waitall` 占 20%”，但“是哪个 rank 在等、等了多久”要靠追踪或 VTune 的时间线。

5 perf 分析要点

拿到 `perf` 报告后，按下面五个问题逐个对照：

1. **哪些函数占用最多时间**：先抓 top-3，确认 `bssn_rhs`、`MPI_Waitall` 等是否如预期出现在报告里。
2. **计算 vs 访存 vs 通信瓶颈**：结合 IPC、cache miss、TLB miss 判断热点函数是 compute-bound、memory-bound 还是 communication-bound。
3. **IPC 是否异常**：IPC 远低于架构峰值（如 ARM 上理论 2 到 4）说明大量停顿。
4. **rank 间负载均衡**：如果 `MPI_Waitall` 或 `MPI_Barrier` 占比偏高，说明某 rank 比别的 rank 工作量多或慢。
5. **OpenMP 线程并行效率**：如果某函数 CPU time 很高但 wall time 不低，可能是线程没铺开或伪共享。

在多 rank 程序里，单看一个 rank 的 `perf` 报告会误导。建议对每个 rank 单独采样（不同 `perf.data` 文件），或用 `perf record -C` 绑定特定 CPU 采样，再横向比较。

6 VTune 工具：V100 节点 host 端分析

Task2 的 ABEGPU 跑在 V100 节点上，host 是 x86 CPU。这里的调用链特别长：Python 驱动 → TwoPuncture → mpirun → ABEGPU → CUDA API。我们用 Intel VTune 分析 host 端，看清这条链上每个环节花了多少时间。

6.1 完整调用链

VTune 能把整条 host 调用链展开。一个典型的调用树看起来像：

```
main (Python 驱动入口)
  TwoPuncture (初始数据生成)
  mpirun (启动多 rank)
    ABEGPU::evolve (BSSN 演化主循环)
      bssn_rhs → CUDA 调用 → cuCtxSynchronize
      MPI_Waitall (ghost zone 交换)
```

Listing 1: ABEGPU host 端调用链示意

6.2 Hotspots 分析

Hotspots 是最常用的 VTune 分析类型，告诉你 host 时间花在了哪些函数。在集群上命令行采样：

```
vtune -collect hotspots -result-dir r0hs -- mpirun -np 4 ./ABEGPU
```

多 rank 程序必须为每个 rank 指定不同结果目录，否则 VTune 会把多个 rank 的数据混在一起。可以用 `mpirun -np 4 sh -c 'rank=$SLURM_PROCID; vtune -collect hotspots -result-dir r$rankhs -- ./ABEGPU'`，或先只采一个 rank 做诊断。

6.3 Bottom-up 调用树

VTune 的 **Bottom-up**（自底向上）视图从最耗时的函数出发，沿调用树向上追溯到调用者。例如你看到 `cuCtxSynchronize` 占 host 时间 35%，展开后能看到它是被 `bssn_rhs` 调的，`bssn_rhs` 又是被 `ABEGPU::evolve` 调的。这样就能回答“这个耗时函数是被谁拖出来的”。

6.4 Flame Graph 与 Threads 视图

Flame Graph（火焰图）把调用栈画成一层层的横向条带，宽度就是时间占比，一眼能看出最宽的那条就是热点。**Threads** 视图按线程排列时间线，能看到 OpenMP 线程是否同时在干活，还是有的线程早早算完在等。

6.5 多 rank 结果下载

VTune 命令行采样生成的是结果目录（如 `r0hs`），需要整体下载到本地用 GUI 打开。GUI 提供的 Flame Graph、Bottom-up、源码视图远比命令行直观：

```
scp -r <user>@<v100-node>:~/r0hs .
```

GUI 显示源码行需要采样时的可执行文件（带 `-g`）。把 `ABEGPU` 一起下载，在 **Tools > Options > Binary/Symbol Search** 里补上它的路径。

7 VTune 回答的问题

把 VTune 的几个视图配合起来，能回答 task2 host 端的四个关键问题：

1. **完整调用链哪些 host 函数耗时**: Bottom-up 给出排名, Flame Graph 给出占比可视化。
2. **MPI / CUDA API / 线程等待占比**: Threads 视图看等待, Hotspots 看 MPI_Waitall、cuCtxSynchronize 的占比。
3. **CPU 是否及时提交 GPU 工作**: 如果 host 在 launch kernel 之后立刻进入长时间同步等待, 说明 CPU 提交工作是及时的, 只是 GPU 还在算; 如果 host 自己在算别的导致 GPU 空闲, 说明提交不及时。
4. **负载不均衡**: rank 间时间线长度不一致, 或某 rank 的 MPI_Waitall 显著长于其它 rank。

8 关键误区: host 等待不等于 kernel 时间

这是 Lab4 最容易踩的坑。在 VTune 上你会看到 cuCtxSynchronize (或 cudaDeviceSynchronize) 占了大量 host 时间, 第一反应往往是“GPU kernel 跑了好久”。但 cuCtxSynchronize 只是 host 线程在等待 GPU 完成, 它占的时间长只说明 host 在等, 并不等于 kernel 真的运行了那么久。

直觉 想象你在餐厅点完菜坐等上菜。你“等了 40 分钟”不等于“厨师做了 40 分钟”, 可能厨师 5 分钟就做完了, 只是你的单子排在后面。cuCtxSynchronize 就是“你坐着等”的时间, 要拆开看厨师到底忙了多久, 必须用 Nsight Systems 看 GPU 时间线。

所以判断 GPU kernel 真实耗时, 必须结合下一章的 Nsight Systems: 在 nsys 的时间线上, kernel 的实际执行区间是连续的一段, 而 host 上的同步等待可能跨越多个 kernel 的发射。host 同步时间和 kernel 时间的关系是:

$$T_{\text{host-sync}} = T_{\text{kernel-actual}} + T_{\text{queue-wait}} + T_{\text{launch-overhead}} \quad (8.1)$$

其中 $T_{\text{queue-wait}}$ 是 kernel 在队列里排队的时间, $T_{\text{launch-overhead}}$ 是发射开销。只有当队列里没有其它 kernel 时, host 同步时间才约等于 kernel 时间。

9 在 AMSS-NCKU 中应用

把上面的方法套到 AMSS-NCKU 上, 推荐的流程是:

1. **先 profile 完整 baseline**: 不要一上来就只看某个猜想的函数。对 task1 跑 perf stat -d 加 perf record --call-graph dwarf, 对 task2 跑 VTune Hotspots, 确认整条调用链都出现在报告里。
2. **确认所有模块都被采到**: 检查报告里既有 TwoPunctureABE (初始数据), 也有 bssn_rhs (演化右端项), 不能只看到一个。
3. **分类瓶颈**: 根据 IPC、cache miss、MPI 占比, 判断当前 baseline 是 compute-bound、memory-bound 还是 communication-bound。
4. **按瓶颈选优化方向**: compute-bound 考虑向量化或 GPU kernel; memory-bound 考虑数据布局、cache blocking; communication-bound 考虑 rank 间负载均衡或减少 ghost exchange。
5. **优化后重新 profile 验证**: 每次优化后重跑同样的 profiler, 确认瓶颈比例确实下降, 而不是被转移。

例 假设我们在 Kunpeng 920B 上对 task1 baseline 跑 perf stat -d 与 perf record, 得到以下结果 (取关键行):

指标	数值
bssn_rhs 占 CPU time	60%
MPI_Waitall 占 CPU time	20%
IPC	0.6
L1-dcache-load-miss	28%
branch-misses	3%

Table 3: task1 baseline perf 结果 (假设)

设总 wall time 为 100 秒, 那么 bssn_rhs 占 60 秒, MPI_Waitall 占 20 秒。

先看 bssn_rhs: IPC 只有 0.6, 而 ARM v8.2 理论 IPC 峰值约 4 (每周期 4 条指令), 停顿占比约为 $1 - 0.6/4 = 85\%$ 。结合 L1-dcache-load-miss 高达 28%, 可以判断 bssn_rhs 虽然名义上是“计算函数”, 但实际瓶颈在访存: 数据没在 L1 缓存里, 流水线在等数据。这是一个 memory-bound 的热点。

再看 MPI_Waitall 占 20%: 说明 rank 间存在负载不均衡或同步等待, 某 rank 先算完在等其它 rank。branch-misses 仅 3%, 不是主要问题。

优化方向因此很清楚: 优先优化 bssn_rhs 的访存模式 (数据布局、cache blocking、向量化), 把它从 memory-bound 推向 compute-bound; 其次处理 rank 间负载不均衡 (均匀划分网格)。如果只盯着 MPI_Waitall 去优化通信库, 是治标不治本, 因为 rank 间等待的根因可能是某 rank 的 bssn_rhs 算得慢。

10 本章你将学会

1. 区分 perf stat、perf record、perf report 三个子命令的用途
2. 区分采样与追踪, 知道什么时候必须用追踪
3. 读取 perf stat 的 IPC、cache miss、TLB miss、branch miss 计数器
4. 用 VTune 的 Hotspots、Bottom-up、Flame Graph、Threads 视图分析 host 端调用链
5. 避免“把 host 同步等待误认为 kernel 时间”的陷阱, 知道要结合 Nsight Systems

11 要点速查

工具 / 视图	回答什么	Lab4 用途
perf stat -d	硬件计数器累计值	task1 IPC、cache miss 体检
perf record --call-graph dwarf	采样热点 + 调用栈	task1 函数级定位
perf report	查看采样报告	task1 交互式浏览
VTune Hotspots	host 时间花在哪	task2 调用链排名
VTune Bottom-up	耗时函数的调用来源	task2 追溯 cuCtxSynchronize
VTune Flame Graph	调用栈占比可视化	task2 一眼定位热点
VTune Threads	线程时间线	task2 OpenMP 效率

Table 4: CPU profiling 工具速查

12 小结

CPU 性能分析的第一步是“不靠猜”。在 AMSS-NCKU 这种调用链横跨 Python、TwoPuncture、MPI、ABE 的程序里，你必须先用 profiler 把完整 baseline 跑一遍，确认每个模块都出现在报告里，再决定优化哪个。task1 在 Kunpeng 920B 上用 perf 采样，关注 IPC、cache miss、rank 间等待；task2 在 V100 节点 host 上用 VTune，关注完整调用链里 host 函数的耗时排名和线程效率。最重要的是不要把 cuCtxSynchronize 的 host 等待时间当成 GPU kernel 时间，那是 host 在等，不是 GPU 在算，kernel 的真实时间要靠下一章的 Nsight Systems 才能看清。

讲义基于 HPC101 2025 Lab4 实验内容编写